

Informe Final – Proyecto de Videojuego: Tiro con Arco Atlantis

Curso: Informática II

Integrantes: Frank Alexis Bedoya Toro y Daniel Elías Ayala Marichal

Fecha de entrega: 05 de junio de 2026

Introducción

El proyecto Tiro con Arco Atlantis fue desarrollado como trabajo final del curso de Informática II con el propósito de integrar los conocimientos adquiridos durante el semestre en una aplicación interactiva y funcional. La temática seleccionada combina el deporte del tiro con arco con un entorno submarino inspirado en la ciudad perdida de Atlantis, permitiendo incorporar diferentes fenómenos físicos propios de este ambiente.

Desde el inicio se buscó diseñar un videojuego que no solamente cumpliera los requisitos académicos establecidos, sino que también ofreciera una experiencia entretenida para el usuario. Para lograrlo, se plantearon dos niveles con dinámicas claramente diferenciadas, donde el jugador debe adaptarse a distintos retos y condiciones del entorno.

A lo largo del desarrollo fue necesario aplicar conceptos de programación orientada a objetos, memoria dinámica, diseño UML, control de versiones mediante Git e implementación de interfaces gráficas utilizando Qt. Adicionalmente, se incorporaron modelos físicos, un agente inteligente y mecanismos de dificultad que permitieran aumentar progresivamente el desafío para el jugador.

El resultado es un videojuego que combina elementos de precisión, estrategia y adaptación al entorno, permitiendo evidenciar de manera práctica la aplicación de los conocimientos desarrollados durante el curso.

Planteamiento del problema

El desarrollo del videojuego buscó resolver la necesidad de integrar en una sola aplicación los diferentes conocimientos adquiridos durante el curso, especialmente aquellos relacionados con programación orientada a objetos, estructuras dinámicas, diseño de interfaces gráficas y simulación física.

La principal dificultad consistía en construir un sistema capaz de representar fenómenos físicos de forma visual e interactiva, manteniendo una arquitectura organizada y reutilizable. Adicionalmente, el proyecto debía incorporar distintos niveles, un agente inteligente, dificultad configurable, interfaz gráfica en Qt y elementos propios de un videojuego funcional.

La solución propuesta fue un simulador de tiro con arco submarino donde el jugador debe adaptarse a distintos entornos y condiciones físicas. En el primer nivel el desafío se centra en

la precisión de los disparos parabólicos bajo resistencia del agua, mientras que en el segundo nivel se introducen corrientes marinas, movimiento libre y un combate dinámico en tiempo real.

Definición general y objetivos

El objetivo general del proyecto fue desarrollar un videojuego funcional utilizando C++ y Qt que permitiera aplicar los conceptos estudiados durante el curso mediante la simulación de un sistema interactivo basado en tiro con arco submarino.

Como objetivos específicos se definieron:

- Implementar al menos tres modelos físicos diferentes dentro del videojuego.
- Utilizar programación orientada a objetos mediante una arquitectura modular basada en clases.
- Incorporar memoria dinámica y relaciones entre objetos.
- Desarrollar una interfaz gráfica utilizando Qt.
- Implementar un agente inteligente con componentes de percepción, razonamiento, acción y aprendizaje.
- Diseñar dos niveles con mecánicas claramente diferenciadas.
- Implementar un sistema de dificultad configurable.
- Gestionar el proyecto mediante repositorios y control de versiones utilizando Git.

Metodología y planificación

El desarrollo del proyecto no siguió una metodología formal estricta como Scrum o Cascada. En la práctica, se trabajó mediante una estrategia incremental basada en módulos, combinada con división de tareas y un proceso constante de prueba y error.

Inicialmente se realizó la contextualización del videojuego y el diseño conceptual de los niveles. Posteriormente se elaboró el diagrama UML de la capa lógica, definiendo las clases principales y las relaciones entre ellas.

Una vez definida la arquitectura, el trabajo se dividió entre los integrantes. Frank Alexis Bedoya Toro se encargó principalmente del diseño UML, implementación de los modelos físicos, desarrollo de la lógica principal del videojuego e integración de la interfaz gráfica. Daniel Elías Ayala Marichal participó principalmente en la implementación de elementos complementarios del sistema, diseño de componentes gráficos y apoyo en la integración general del proyecto.

Durante el desarrollo se realizaron múltiples pruebas parciales para verificar el funcionamiento de las físicas, la interacción entre objetos y la correcta comunicación entre la lógica y la interfaz gráfica.

Diseño general de la solución

La arquitectura del videojuego se dividió en dos capas principales:

La primera corresponde a la capa lógica, encargada de gestionar las físicas, personajes, proyectiles, colisiones, sistema de puntuación, temporizadores y comportamiento del agente inteligente.

La segunda corresponde a la capa gráfica implementada mediante Qt, responsable de representar visualmente todos los elementos del videojuego y permitir la interacción con el usuario.

El diseño orientado a objetos permitió separar responsabilidades y facilitar el mantenimiento del código. Cada clase cumplía una función específica dentro del sistema, evitando dependencias innecesarias y favoreciendo la reutilización de componentes.

Para cumplir los requisitos académicos también se implementó una jerarquía de herencia propia relacionada con los modelos físicos utilizados en los distintos niveles.

Desarrollo e implementación

La implementación del videojuego se apoyó en varios modelos físicos distintos.

El primero corresponde al movimiento parabólico de los proyectiles, utilizado como base para las flechas disparadas por los personajes. Este movimiento se modeló mediante ecuaciones cinemáticas que consideran velocidad inicial, ángulo de lanzamiento y gravedad.

El segundo modelo físico corresponde a la resistencia del agua, utilizada para disminuir progresivamente la velocidad de los proyectiles y generar una sensación más coherente con el entorno submarino.

El tercer modelo físico se implementó en el nivel denominado Abismo Oceánico, donde las corrientes marinas generan una aceleración horizontal adicional que modifica la trayectoria de las flechas durante su desplazamiento.

Adicionalmente se incorporó un movimiento oscilatorio para el enemigo mediante funciones trigonométricas. Este comportamiento permite generar trayectorias parcialmente impredecibles que incrementan la dificultad del juego sin llegar a ser imposibles de anticipar.

Uno de los aspectos más importantes fue el desarrollo del agente inteligente. Este agente percibe la posición del jugador y las condiciones del entorno, analiza posibles acciones y selecciona respuestas apropiadas. Aunque se trata de una inteligencia sencilla, cumple con los componentes exigidos por el proyecto: percepción, razonamiento, acción y almacenamiento de información para mejorar su comportamiento durante la partida.

Problemas enfrentados durante el desarrollo

Si tuviéramos que identificar la parte más compleja del proyecto, probablemente sería la integración entre la lógica física y la representación gráfica.

Aunque muchas funciones funcionaban correctamente de manera independiente, en varias ocasiones aparecieron errores al momento de sincronizar la posición lógica de los objetos con su representación visual dentro de Qt. Esto ocasionó situaciones donde proyectiles, personajes o enemigos existían correctamente dentro de la simulación, pero no aparecían en pantalla o se mostraban en posiciones incorrectas.

Otro problema importante estuvo relacionado con la implementación de la dificultad. Inicialmente se pensó en modificar únicamente variables simples, pero posteriormente se comprendió que el requisito exigía una abstracción más profunda. Esto obligó a replantear parte de la lógica del enemigo, sus patrones de movimiento y la manera en que interactuaba con el jugador.

También surgieron múltiples dificultades asociadas al agente inteligente. Ajustar correctamente los ángulos de disparo, la toma de decisiones y el comportamiento dinámico del enemigo requirió numerosas pruebas y correcciones.

La integración mediante Git generó varios conflictos entre ramas debido a que distintos módulos se desarrollaban simultáneamente. En diferentes momentos fue necesario resolver conflictos manualmente y reorganizar partes del código para evitar inconsistencias.

Finalmente, durante las etapas finales del proyecto aparecieron errores relacionados con la visualización de sprites, la carga de recursos y la actualización gráfica de la escena. Muchos de estos problemas fueron corregidos mediante pruebas sucesivas y revisión detallada de la arquitectura implementada.

Resultados y discusión

El resultado obtenido fue un videojuego funcional que cumple la mayoría de los requerimientos establecidos en la guía del curso. Se logró implementar una arquitectura orientada a objetos, múltiples modelos físicos, un sistema de dificultad configurable, un agente inteligente y una interfaz gráfica basada en Qt.

Los dos niveles presentan diferencias claras tanto en la perspectiva como en la jugabilidad. El primero ofrece una experiencia más estratégica basada en precisión y cálculo, mientras que el segundo introduce una dinámica más rápida y exigente debido a las corrientes marinas y al movimiento continuo del enemigo.

A pesar de los avances logrados, la etapa final evidenció algunas limitaciones relacionadas principalmente con la integración gráfica y ciertos detalles de visualización. Sin embargo, estas dificultades permitieron comprender mejor la complejidad que implica conectar correctamente la lógica de un videojuego con su representación visual.

Conclusiones

El desarrollo de Tiro con Arco Atlantis permitió aplicar de manera práctica gran parte de los conocimientos adquiridos durante el curso de Informática II. Más allá de la programación

individual de funciones o clases, el proyecto exigió comprender cómo interactúan múltiples componentes dentro de un sistema complejo.

Durante el proceso se fortalecieron habilidades relacionadas con programación orientada a objetos, modelado físico, diseño de interfaces gráficas, trabajo colaborativo y control de versiones.

Una de las principales enseñanzas fue comprender que los mayores retos no suelen encontrarse en la implementación aislada de algoritmos, sino en la integración de todos los componentes dentro de una aplicación completa. Asimismo, la experiencia permitió valorar la importancia de una buena planificación inicial, un diseño UML adecuado y una estrategia de pruebas constante durante todo el desarrollo.

En términos generales, consideramos que el proyecto cumplió su propósito académico y representó una experiencia significativa de aprendizaje en el desarrollo de software interactivo utilizando C++ y Qt.

Cambios realizados entre los momentos I, II y III

Durante el desarrollo del proyecto fue necesario realizar diversos ajustes respecto a las propuestas iniciales planteadas en los momentos anteriores.

En el Momento I se definió la temática general del videojuego, los dos niveles principales y las físicas que se pretendían implementar. Sin embargo, varias de estas ideas se encontraban aún a nivel conceptual y requerían mayor detalle para su implementación.

Durante el Momento II se construyó el diagrama UML de la capa lógica y se definieron con mayor precisión las responsabilidades de cada clase. En esta etapa también se profundizó en la arquitectura del agente inteligente y en la organización de las físicas dentro del sistema.

Finalmente, en el Momento III se realizaron ajustes importantes relacionados con la integración gráfica, el sistema de dificultad y el comportamiento del agente inteligente. Algunas clases fueron modificadas respecto al diseño inicial para facilitar la comunicación entre la capa lógica y la capa gráfica. También se realizaron cambios en el manejo de proyectiles, actualización de escenas y control de colisiones, como resultado de los problemas encontrados durante las pruebas.

Estos cambios permitieron obtener una versión más estable y funcional del videojuego respecto a las propuestas planteadas inicialmente.

Guía básica de instalación y uso

Requisitos

- Sistema operativo Windows.
- Qt Creator.

- Compilador MinGW compatible con Qt.
- Bibliotecas necesarias incluidas dentro del proyecto.

Instalación

1. Clonar o descargar el repositorio del proyecto.
2. Abrir el archivo .pro desde Qt Creator.
3. Configurar el kit de compilación correspondiente.
4. Compilar el proyecto utilizando la opción Build.
5. Ejecutar el programa desde Qt Creator o mediante el ejecutable generado.

Uso

1. Iniciar el videojuego.
2. Seleccionar el nivel deseado.
3. Utilizar los controles definidos para mover el personaje y realizar disparos.
4. Adaptar la estrategia a las condiciones físicas de cada nivel.
5. Obtener la mayor cantidad de impactos posible antes de finalizar la partida.

Referencias

- Documentación oficial de C++.
- Documentación oficial de Qt Framework.
- Material de clase suministrado durante el curso de Informática II.
- Documentación oficial de Git y GitHub.
- Apuntes de cinemática y movimiento parabólico utilizados para el modelado físico.
- Recursos gráficos y sonoros libres de derechos utilizados durante el desarrollo.